

What is the DOM?

The **DOM (Document Object Model)** is a representation of an HTML document that the browser creates as a **tree of objects**.

JavaScript can use the DOM to **read, change, add, or remove HTML elements**.

1. 🔥🔥🔥 What is the DOM?

DOM stands for **Document Object Model**.

It represents the HTML page as a **tree of objects** so JavaScript can interact with the page.

Simple Example

```
<body>
  <h1>Hello</h1>
  <p>Welcome</p>
</body>
```

The browser represents it roughly like:

```
Document
 |
body
 /  \
h1    p
```

JavaScript can then access and modify these elements.

```
document.querySelector("h1").textContent = "Hello Ajay";
```

Remember:

DOM → HTML page represented as a tree of objects that JavaScript can interact with.

2. 🔥🔥🔥 How does the browser create the DOM?

The browser reads the **HTML document** and converts its HTML elements into a **DOM tree**.

Simple Example

```
<html>
  <body>
```

```
<h1>Hello</h1>
<p>Welcome</p>
</body>
</html>
```

The browser creates a tree like:

```
Document
|
html
|
body
/  \
h1   p
```

The basic process is:

```
HTML
↓
HTML Parsing
↓
DOM Tree
↓
JavaScript can access the DOM
```

Remember:

Browser reads HTML → parses it → creates the DOM tree.

🔥 🔥 What is the difference between DOM and BOM?

DOM (Document Object Model) represents the **HTML document** and allows JavaScript to interact with page elements.

BOM (Browser Object Model) represents the **browser environment** and allows JavaScript to interact with browser features.

Simple Example

```
// DOM → works with the webpage
document.querySelector("h1").textContent = "Hello";

// BOM → works with the browser
console.log(window.innerWidth);
console.log(window.location.href);
```

DOM	BOM
Represents the HTML document	Represents the browser environment
Works with webpage elements	Works with browser features
Main object → <code>document</code>	Main object → <code>window</code>
Example → <code>document.querySelector()</code>	Example → <code>window.location</code>

Remember:

DOM → **webpage**

BOM → **browser**

🔥 🔥 `querySelector()` vs `getElementById()`?

Both are used to **select HTML elements from the DOM**, but they work differently.

- `getElementById()` → selects an element by its **id**.
- `querySelector()` → selects the **first element that matches a CSS selector**.

Simple Example

```
// getElementById()
const heading = document.getElementById("heading");

// querySelector()
const heading = document.querySelector("#heading");
```

Both select:

```
<h1 id="heading">Hello</h1>
```

Main Difference

<code>getElementById()</code>	<code>querySelector()</code>
Selects by id only	Uses any CSS selector
<code>getElementById("heading")</code>	<code>querySelector("#heading")</code>
Returns the element or null	Returns the first matching element or null

Example with CSS Selector

```
const heading = document.querySelector(".heading");
```

```
const button = document.querySelector("button");

const input = document.querySelector("input[type='text']");
```

Remember:

`getElementById()` → select by ID

`querySelector()` → select using CSS selector

1. 🔥🔥🔥 What is Event Capturing?

Event Capturing is the phase where an event travels **from the outermost element toward the element where the event actually happened**.

In simple words, the event moves **from parent** → **child** → **target element**.

Simple Example

```
<div id="parent">
  <button id="child">Click Me</button>
</div>
```

```
const parent = document.getElementById("parent");
const child = document.getElementById("child");

parent.addEventListener(
  "click",
  () => {
    console.log("Parent");
  },
  true,
);

child.addEventListener("click", () => {
  console.log("Button");
});
```

If we click the button:

```
Parent
Button
```

Here, the event first travels from the parent toward the button. This is called **Event Capturing**.

The `true` in `addEventListener()` tells JavaScript to run the handler during the **capturing phase**.

2. 🔥🔥🔥 What is Event Delegation?

Event Delegation is a technique where we **add one event listener to a parent element instead of adding separate listeners to each child element**.

The parent handles events that happen on its children.

Simple Example

```
<ul id="list">
  <li>Apple</li>
  <li>Banana</li>
  <li>Mango</li>
</ul>
```

Instead of doing this:

```
document.querySelectorAll("li").forEach((item) => {
  item.addEventListener("click", () => {
    console.log(item.textContent);
  });
});
```

We can add one listener to the parent:

```
const list = document.getElementById("list");

list.addEventListener("click", (event) => {
  console.log(event.target.textContent);
});
```

Now, when we click any `<li>`, the parent `<ul>` handles the event.

```
Click Apple
  ↓
<li> Apple
  ↓
<ul> handles the event
```

This works because the event **bubbles from the child toward the parent**.

3. 🔥🔥🔥 Why is Event Delegation Useful?

Event Delegation is useful because we don't need to create a separate event listener for every child element.

1. Fewer Event Listeners

Instead of:

```
item1.addEventListener("click", handler);
item2.addEventListener("click", handler);
item3.addEventListener("click", handler);
```

We can use one listener:

```
parent.addEventListener("click", handler);
```

This makes the code simpler and easier to manage.

2. Works with Dynamically Added Elements

Suppose we add a new `<li>` later:

```
list.innerHTML += "<li>Orange</li>";
```

The parent listener can still handle the click on the new `<li>`.

```
list.addEventListener("click", (event) => {
  console.log(event.target.textContent);
});
```

We don't need to add a new event listener to `Orange`.

3. Better for Large Lists

If there are hundreds of buttons or list items, adding a separate listener to every element is unnecessary.

Event Delegation lets us use **one parent listener** to handle them.

Interview Point

Event Capturing: Event travels from **parent** → **target**.

Event Delegation: We use a **parent's event listener to handle events from its child elements**.

🔥 🔥 `preventDefault()` vs `stopPropagation()`

These two methods do **different things**:

- `preventDefault()` → stops the browser's **default action**.
 - `stopPropagation()` → stops the event from **moving to parent/child elements**.
-

1. 🔥 What does `preventDefault()` do?

`preventDefault()` stops the **default behavior provided by the browser**.

Simple Example

Suppose we have a link:

```
<a href="https://example.com" id="link">Visit Website</a>
```

Normally, clicking the link opens the website.

We can stop that default behavior:

```
const link = document.getElementById("link");

link.addEventListener("click", (event) => {
  event.preventDefault();

  console.log("Link clicked");
});
```

Now the link is clicked, but the browser **does not open the website**.

Common Uses

`preventDefault()` is commonly used for:

- Stopping a form from submitting
- Stopping a link from navigating
- Preventing default browser actions

Example:

```
form.addEventListener("submit", (event) => {
  event.preventDefault();

  console.log("Form submission stopped");
});
```

2. 🔥 What does `stopPropagation()` do?

`stopPropagation()` stops the event from **propagating between elements**.

In simple words, it prevents the event from continuing to the parent or child during the event flow.

Simple Example

```
<div id="parent">
  <button id="child">Click Me</button>
</div>
```

```
const parent = document.getElementById("parent");
const child = document.getElementById("child");

parent.addEventListener("click", () => {
  console.log("Parent clicked");
});

child.addEventListener("click", (event) => {
  event.stopPropagation();

  console.log("Button clicked");
});
```

When we click the button:

```
Button clicked
```

The "Parent clicked" message does **not** appear.

Why?

Because:

```
event.stopPropagation();
```

stops the event from reaching the parent.

Main Difference

<code>preventDefault()</code>	<code>stopPropagation()</code>
Stops the browser's default action	Stops event propagation
Does not stop event bubbling by itself	Does not stop the browser's default action

<code>preventDefault()</code>	<code>stopPropagation()</code>
Example: stop link navigation	Example: stop event reaching parent
Used for browser behavior	Used for event flow

Easy Way to Remember

```
preventDefault()
  ↓
"Browser, don't do your normal action."

stopPropagation()
  ↓
"Event, don't travel any further."
```

Interview Answer

`preventDefault()` prevents the browser's default behavior, while `stopPropagation()` prevents the event from propagating to other elements.

🔥 🔥 What is `stopImmediatePropagation()`?

`stopImmediatePropagation()` stops the event from **propagating to other elements** and also stops **other event listeners on the same element** from running.

In simple words:

`stopPropagation()` → event ko parent tak jaane se rokta hai. `stopImmediatePropagation()` → parent tak jaane se **aur same element ke baaki listeners ko bhi rokta hai**.

Simple Example

```
<button id="btn">Click Me</button>
```

Suppose the same button has **multiple event listeners**:

```
const btn = document.getElementById("btn");

btn.addEventListener("click", () => {
  console.log("Listener 1");
});

btn.addEventListener("click", (event) => {
  event.stopImmediatePropagation();

  console.log("Listener 2");
});
```

```
});  
  
btn.addEventListener("click", () => {  
  console.log("Listener 3");  
});
```

When we click the button:

```
Listener 1  
Listener 2
```

Listener 3 will **not run**.

Why?

Because:

```
event.stopPropagation();
```

stops the event **immediately**, so other listeners on the same element don't get a chance to run.

stopPropagation() vs stopImmediatePropagation()

```
stopPropagation()  
  ↓  
Stops event from reaching other elements  
  ↓  
But other listeners on the same element can still run  
  
stopImmediatePropagation()  
  ↓  
Stops event propagation  
  +  
Stops other listeners on the same element
```

Easy Example to Remember

Imagine a button has **3 security guards**:

```
Button  
├─ Guard 1
```

```
├ Guard 2
└ Guard 3
```

If Guard 2 uses `stopPropagation()`:

```
Guard 1 → runs
Guard 2 → runs
Guard 3 → can still run
Parent → event doesn't continue
```

If Guard 2 uses `stopImmediatePropagation()`:

```
Guard 1 → runs
Guard 2 → runs
Guard 3 → ❌ doesn't run
Parent → ❌ event doesn't continue
```

Interview Answer

`stopImmediatePropagation()` stops the event from propagating further and also prevents other event listeners attached to the same element from executing.

Quick Difference

Method	Stops Parent Propagation	Stops Other Listeners on Same Element
<code>stopPropagation()</code>	✅ Yes	❌ No
<code>stopImmediatePropagation()</code>	✅ Yes	✅ Yes

1. 🔥🔥 What are Browser Events?

Browser Events are **actions or occurrences in the browser that JavaScript can detect and respond to**.

In simple words, when something happens on a webpage, the browser creates an **event**.

Common Examples

```
User clicks a button      → click event
User types in an input    → input event
User submits a form       → submit event
User moves the mouse      → mousemove event
Webpage finishes loading  → load event
```

JavaScript can listen for these events using `addEventListener()`.

Simple Example

```
<button id="btn">Click Me</button>
```

```
const button = document.getElementById("btn");

button.addEventListener("click", () => {
  console.log("Button was clicked");
});
```

When the user clicks the button, the browser creates a **click event**, and JavaScript runs the event handler.

```
User clicks button
      ↓
Browser creates click event
      ↓
JavaScript detects the event
      ↓
Event handler runs
```

Interview Answer

Browser events are actions or occurrences detected by the browser, such as clicks, typing, form submission, and page loading, which JavaScript can respond to.

2. 🔥🔥 What is the difference between `target` and `currentTarget`?

Both `target` and `currentTarget` are properties of the **event object**, but they refer to different elements.

- `event.target` → the element where the event **actually happened**.
- `event.currentTarget` → the element whose **event listener is currently running**.

Simple Example

```
<div id="parent">
  <button id="child">Click Me</button>
</div>
```

```
const parent = document.getElementById("parent");
```

```
parent.addEventListener("click", (event) => {  
  console.log("target:", event.target);  
  console.log("currentTarget:", event.currentTarget);  
});
```

If we click the button:

```
target      → button  
currentTarget → div
```

Why?

Because the **button was actually clicked**, so:

```
event.target;
```

points to the button.

But the event listener is attached to the `<div>`, so:

```
event.currentTarget;
```

points to the `<div>`.

Visual Example

```
<div id="parent">          ← currentTarget  
  ↓  
<button id="child">       ← target  
  Click  
</button>  
</div>
```

Important Point

`target` can be a **child element**, while `currentTarget` is the element on which the **currently executing listener is attached**.

This is especially important in **Event Delegation**.

Easy Way to Remember

```
target
  ↓
"Actually kis element par event hua?"

currentTarget
  ↓
"Kis element ka listener abhi run ho raha hai?"
```

Interview Answer

event.target refers to the element where the event originally occurred, while **event.currentTarget** refers to the element whose event listener is currently handling the event.

1. 🔥🔥 What is Reflow?

Reflow happens when the browser has to **recalculate the size and position of elements on the webpage** because something changed.

In simple words:

Layout change → Browser calculates the layout again → Reflow

Simple Example

Suppose we have:

```
<div id="box">Hello</div>
```

```
const box = document.getElementById("box");

box.style.width = "500px";
```

The width of the element changed.

Now the browser may need to recalculate:

```
Width
Height
Position
Spacing
Other affected elements
```

This recalculation is called **Reflow**.

Another Example

```
box.style.display = "none";
```

When an element is removed from the layout, other elements may need to move.

So the browser recalculates the layout again.

Common Things That Can Cause Reflow

```
Change width/height  
Change margin/padding  
Add or remove an element  
Change display  
Change font size  
Change position
```

Important Point

Reflow can be **expensive** because the browser may need to recalculate the layout of many elements.

```
DOM/CSS change  
  ↓  
Layout changes  
  ↓  
Browser recalculates positions and sizes  
  ↓  
Reflow
```

2. 🔥 What is Repaint?

Repaint happens when the browser needs to **draw an element again because its visual appearance changed**, but its size or position did not change.

In simple words:

Visual change → Browser draws it again → Repaint

Simple Example

```
<div id="box">Hello</div>
```

```
const box = document.getElementById("box");  
  
box.style.color = "red";
```

The text color changes, but the element's:

```
Width  
Height  
Position
```

do not change.

So the browser only needs to **draw the element again**.

That is called **Repaint**.

Common Things That Can Cause Repaint

```
Change color  
Change background-color  
Change visibility  
Change box-shadow  
Change outline
```

Reflow vs Repaint

```
Reflow  
↓  
Layout changes  
↓  
Size/position needs recalculation  
  
Repaint  
↓  
Appearance changes  
↓  
Element needs to be drawn again
```

Easy Example to Remember

Imagine a room:


```
Move the furniture
  ↓
Room layout needs to be calculated again
  ↓
Reflow

Paint the furniture a different color
  ↓
Only appearance changes
  ↓
Repaint
```

Interview Answer

Reflow occurs when the browser recalculates the layout of elements after a change in their size or position. Repaint occurs when the browser redraws elements after a visual change that does not affect their layout.

Important Relationship

A change that causes **Reflow will generally also require Repaint**, because after recalculating the layout, the browser needs to draw the updated result.

But a **Repaint does not necessarily cause Reflow**.

🔥 What is Debouncing?

Debouncing is a technique that delays the execution of a function until a certain amount of time has passed after the last event.

In simple words:

Jab event baar-baar trigger ho raha ho, function ko har baar run nahi karna. Last event ke baad thoda wait karo, phir function run karo.

Why do we need Debouncing?

Suppose we have a search box:

```
<input id="search" placeholder="Search..." />
```

User types:

```
A
Aj
```

```
Aja  
Ajay
```

Every time the user types a character, an **input** event occurs.

Without debouncing:

```
A    → API call  
Aj   → API call  
Aja  → API call  
Ajay → API call
```

So, just typing **Ajay** can make **4 API calls**.

This is unnecessary.

With debouncing:

```
A  
Aj  
Aja  
Ajay  
    ↓  
Wait  
    ↓  
User stopped typing  
    ↓  
API call
```

Now only **one API call** is made.

How does Debouncing work?

Suppose we set the delay to **500ms**.

Every time the user types something:

1. Start a timer.
2. If another event happens before 500ms, cancel the previous timer.
3. Start a new timer.
4. When the user stops typing for 500ms, run the function.

Example

```
let timer;
```

```
function searchUser(value) {  
  console.log("Searching for:", value);  
}  
  
const search = document.getElementById("search");  
  
search.addEventListener("input", (event) => {  
  clearTimeout(timer);  
  
  timer = setTimeout(() => {  
    searchUser(event.target.value);  
  }, 500);  
});
```

Let's understand this step by step

Suppose the user types:

A

Timer starts:

A → wait 500ms

Before 500ms, user types:

Aj

The previous timer is cancelled:

A →  cancelled

Aj → wait 500ms

Then user types:

Aja

Again:

Aj →  cancelled

Aja → wait 500ms

Finally user types:

Ajay

No more typing happens.

After 500ms:

```
Ajay
  ↓
500ms wait
  ↓
searchUser("Ajay")
```

So the function runs **only once after the user stops typing**.

Real-World Uses of Debouncing

Debouncing is commonly used for:

1. Search Box

```
User typing
  ↓
Wait until typing stops
  ↓
API call
```

2. Auto-save

```
User keeps editing
  ↓
Wait until user stops
  ↓
Save data
```

3. Window Resize

When the user continuously resizes the browser:

```
Resize → Resize → Resize → Resize → Resize
                ↓
            User stops
                ↓
          Run function
```

Instead of running expensive code after every resize event, we wait until resizing stops.

Debouncing vs Normal Event

Without Debouncing:

```
Event → Function
Event → Function
Event → Function
Event → Function
Event → Function
```

With Debouncing:

```
Event
Event
Event
Event
Event
  ↓
Wait
  ↓
Function
```

Easy Way to Remember

Think about a **search box**.

You don't want to search the server after **every single letter**.

You want to say:

"I'll wait until the user stops typing, then I'll search."

That's **Debouncing**.

Interview Answer

Debouncing is a technique where a function is executed only after a specified delay has passed since the last event. If the event happens again before the delay finishes, the previous timer is reset.

One-Line Memory Trick

```
Debouncing = "Wait until the events stop, then run the function."
```

🔥 🔥 🔥 What is Throttling?

Throttling is a technique that limits how often a function can execute when an event happens repeatedly.

In simple words:

Event baar-baar trigger ho, lekin function ko har baar run nahi karna. Function ko ek fixed time interval par maximum ek baar run karna.

Why do we need Throttling?

Suppose we are listening to the `scroll` event.

When the user scrolls, the browser can generate **many scroll events very quickly**:

```
Scroll → Scroll → Scroll → Scroll → Scroll → Scroll → Scroll
```

If we run an expensive function for every event:

```
Scroll → Function  
Scroll → Function  
Scroll → Function  
Scroll → Function  
Scroll → Function  
Scroll → Function  
Scroll → Function
```

This can make the webpage slow because the function is running too frequently.

With throttling, we can say:

"Function maximum har 500ms mein ek baar chalega."

```
Scroll → Scroll → Scroll → Scroll → Scroll  
↓  
Function runs  
↓  
wait 500ms  
↓
```

```
Function runs
↓
wait 500ms
↓
Function runs
```

How does Throttling work?

Suppose our throttle time is **500ms**.

```
0ms      → Function runs  ✓
100ms    → ✗
200ms    → ✗
300ms    → ✗
400ms    → ✗
500ms    → Function runs  ✓
600ms    → ✗
700ms    → ✗
800ms    → ✗
900ms    → ✗
1000ms   → Function runs  ✓
```

Even if the event happens hundreds of times, the function will run **at most once every 500ms**.

Simple Example

```
<div id="box"></div>
```

```
let lastRun = 0;

window.addEventListener("scroll", () => {
  const now = Date.now();

  if (now - lastRun >= 500) {
    console.log("Function executed");

    lastRun = now;
  }
});
```

Here:

```
now - lastRun >= 500;
```

means:

Has at least 500ms passed since the function last ran?

If yes → function runs.

If no → function doesn't run.

Real-World Uses of Throttling

1. Scroll Event

```
User scrolls continuously
      ↓
Many scroll events
      ↓
Throttle
      ↓
Function runs at controlled intervals
```

Useful for things like:

- Detecting scroll position
- Showing/hiding a header
- Updating scroll progress

2. Mouse Movement

Mouse movement can generate many events:

```
mousemove → mousemove → mousemove → mousemove
```

If we don't need to process every single movement, throttling can limit how often our function runs.

3. Window Resize

During continuous resizing:

```
Resize → Resize → Resize → Resize → Resize
```


Throttling can make sure the expensive calculation doesn't run too frequently.

🔥 Throttling vs Debouncing

This is a **very important interview difference**.

Debouncing

Wait until the events stop, then run the function.

```
Event → Event → Event → Event
                        ↓
                      STOP
                        ↓
                      WAIT
                        ↓
                    Function runs
```

Example:

Search box

User types continuously → wait → API call.

Throttling

Run the function at a controlled interval while events are happening.

```
Event → Event → Event → Event → Event
    ↓
  Function
    ↓
   500ms
    ↓
  Function
    ↓
   500ms
    ↓
  Function
```

Example:

Scroll

User keeps scrolling → function keeps running, but only at controlled intervals.

Easy Way to Remember

Imagine you're drinking water from a bottle:

Debouncing:

"Jab tak tum peena band nahi karoge, main wait karunga. Band karne ke baad ek baar action lunga."

Throttling:

"Tum kitni bhi baar bolo, main maximum har 500ms mein ek baar action lunga."

One-Line Memory Trick

Debouncing = Events stop → then run

Throttling = Events continue → run at fixed intervals

Interview Answer

Throttling is a technique that limits how frequently a function can execute during repeated events. For example, a throttled function can be allowed to run at most once every 500ms, even if the event occurs many times during that period.

🔥 🔥 🔥 Debouncing vs Throttling?

Both **Debouncing** and **Throttling** are techniques used to control how frequently a function runs when an event happens repeatedly.

The main difference is **when the function is allowed to run**.

🔥 Debouncing

Debouncing waits until the events stop happening, and then runs the function after a specified delay.

In simple words:

"Event baar-baar aa raha hai → wait karo → jab event ruk jaye → function chalao."

Example: Search Box

User types:

A → Aj → Aja → Ajay

With debouncing:

```
A
↓
Aj
↓
Aja
↓
Ajay
↓
User stops typing
↓
Wait 500ms
↓
API call ✓
```

So, if the user keeps typing, the timer keeps getting reset.

Function runs only once after the user stops typing.

Best Use Cases

```
Search input
Auto-save
Form validation
API calls based on user input
```

🔥 Throttling

Throttling allows a function to run at most once during a fixed time interval, even if the event keeps happening.

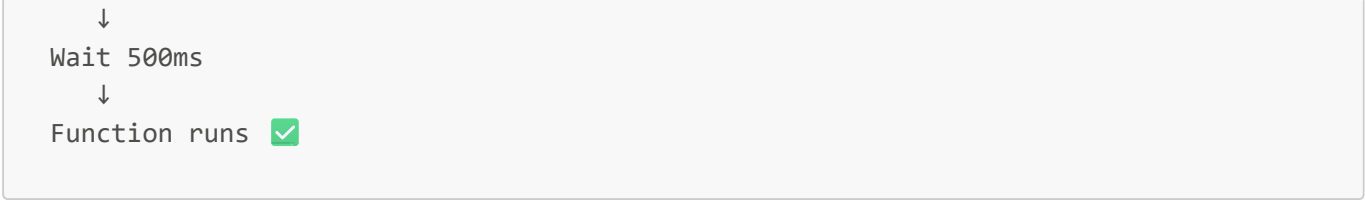
In simple words:

"Event baar-baar aa raha hai → function ko fixed interval par run karte raho."

Example: Scroll

Suppose the throttle interval is 500ms.

```
Scroll → Scroll → Scroll → Scroll
↓
Function runs ✓
↓
Wait 500ms
↓
Function runs ✓
```



The user is still scrolling, but the function doesn't run for every scroll event.

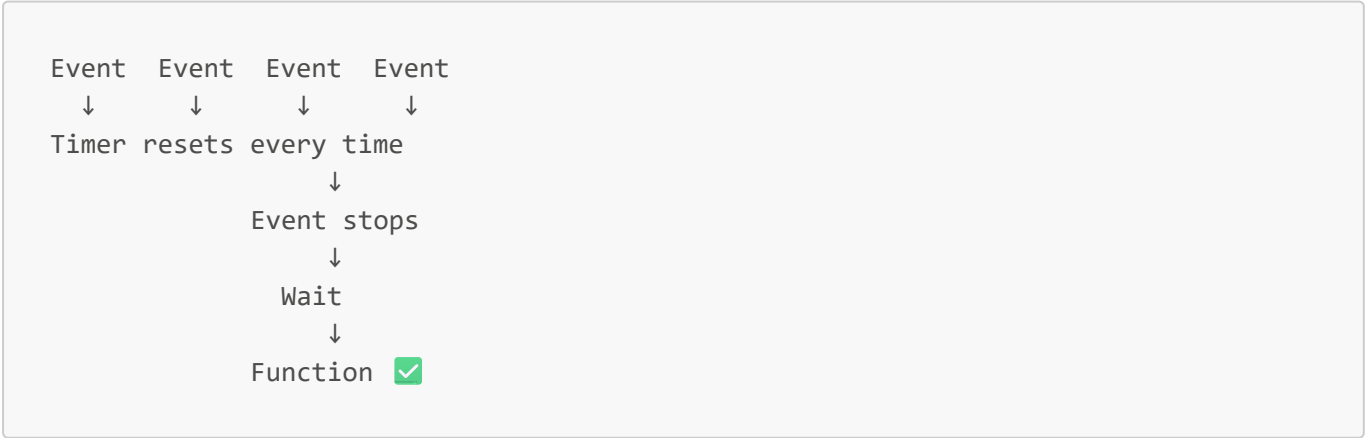
Best Use Cases

- Scroll events
Mouse movement
Window resize
Continuous UI events

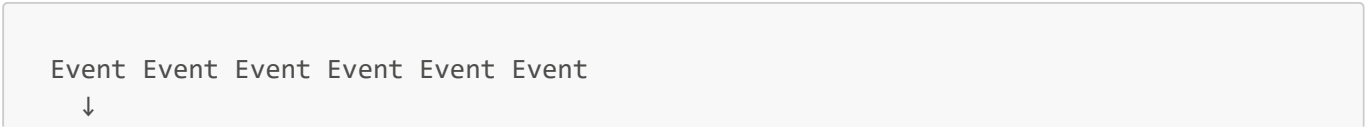
🔥 Debouncing	🔥 Throttling
Waits for events to stop	Works while events continue
Runs after the last event	Runs at fixed intervals
Good for search input	Good for scrolling
Prevents repeated execution after rapid events	Limits execution frequency
Example: API call after typing stops	Example: update UI while scrolling

Easy Visual Difference

Debouncing



Throttling



Function ✓

↓

Wait 500ms

↓

Function ✓

↓

Wait 500ms

↓

Function ✓

🔥 Real-Life Example

Imagine you're at a **restaurant** and you're ordering food.

Debouncing

You keep changing your order:

Pizza

↓

Pizza + Coke

↓

Pizza + Coke + Burger

↓

Pizza + Coke + Burger + Fries

The waiter waits until you **stop changing your order**.

Then:

You stop

↓

Wait

↓

Final order is placed ✓

That's **Debouncing**.

Throttling

Imagine a traffic signal allowing cars to pass **every 10 seconds**.

Even if hundreds of cars are waiting:

```
Cars keep coming
  ↓
Allow cars
  ↓
10 seconds
  ↓
Allow cars
  ↓
10 seconds
  ↓
Allow cars
```

Cars continue coming, but passage is controlled at a fixed interval.

That's **Throttling**.

🔥 Interview Answer

Debouncing delays function execution until the events stop occurring for a specified period, while **throttling** limits the function to execute at most once within a specified time interval.

One-Line Memory Trick

```
Debouncing → "Wait until it stops."
Throttling → "Control how often it runs."
```